

Documentación de Práctica Cafetería más Barista

Andrea Sofía Pais Dos Santos

December 3, 2025

Índice

1. Descripción del problema	página 3
2. Requisitos funcionales	página 3
3. Requisitos no funcionales	página 3
4. Casos de uso	página 4
5. Historias de usuario	página 4
6. Tecnologías Utilizadas	página 4
7. Patrones de código utilizados	página 5
8. Clases e Interfaz	página 5

1 Descripción del problema

Tenemos una cafetería que necesita organizar a los camareros y se repartan los clientes de la forma más óptima posible. Para ello creamos un programa que gestione la entrada de clientes de forma manual y los reparta correctamente a los distintos camareros. Después los camareros al tomar nota de los pedidos, se lo pasa al barista que se va a encargar de producir los cafés y el camarero de repartirlos a los clientes. Luego crear una pequeña aplicación que refleje el funcionamiento del programa, tiene que ser capaz de:

1. Al ejecutar el programa deben estar los camareros y el barista listos empezando a producir cafés y poder manualmente añadir los clientes para que entren cuando yo quiera.
2. Esos clientes por orden de llegada, son atendidos por el camarero que esté disponible.
3. Si el camarero atiende al cliente dentro del tiempo de espera del cliente, el mismo se va contento de la cafetería. Si al camarero no le ha dado tiempo de entregar el café en el tiempo de espera, el cliente se marcha insatisfecho.
4. Y la posibilidad de cerrar la cafetería cuando me apetezca.

2 Requisitos funcionales

Los requisitos funcionales que va a tener el sistema son:

RF01: Cada cliente debe esperar su café durante el tiempo especificado.

RF02: Los clientes deben poder irse si se acaba el tiempo de espera puesto.

RF03: Los camareros deben atender clientes en orden de llegada y deben de continuar trabajando si un cliente se va.

RF04: Gestionar pedidos concurrentemente.

RF05: El barista crea constantemente cafés y los añade a una cola, y el camarero cuando tenga pedidos, retira de la cola los cafés uno a uno que necesite.

RF06: Mostrar estado de clientes, camareros, el barista y el buffer.

3 Requisitos no funcionales

Los requisitos no funcionales que va a tener el sistema son:

RNF01: Mensajes informativos, comprensibles y actualizados.

RNF02: En cada ejecución el proceso debe ser distinto, menos el barista .

RNF03: Los hilos deben finalizar correctamente.

RNF04: Interfaz intuitiva y fácil de usar.

4 Casos de uso

Descripción de casos de uso principales de la aplicación:

Caso de Uso	Descripción
Llegada de Cliente	El cliente llega y espera su café en un tiempo límite. Si se agota el cliente se va sino recibe su pedido.
Preparar Café	El camarero atiende pedidos en orden de llegada, el camarero recoge un café de la cola de cafés preparados por el barista.
Gestionar Cafés	El café está hecho por el barista que se añade a una cola, si cuando el camarero recoge el café y no hay cafés, el cliente se va a ir insatisfecho.

5 Historias de usuario

Identificación	Nombre	Tarea	Objetivo
HU1	Cliente	El cliente llega a la cafetería, pide un café y tiene un tiempo de espera	Pedir un café y esperar por el
HU3	Cliente	El cliente tiene que esperar (tiempo de espera) y si no tiene el café en ese tiempo se marcha	Ser atendido
HU4	Camareros	Gestionar los clientes que van a llegar	Atender pedidos
HU5	Camareros	Cada camarero tiene que comprobar que el cliente no está siendo atendido por otro camarero antes de atenderlo, si fue atendido pues pasar al siguiente	Atender a clientes que no son atendidos

6 Tecnologías Utilizadas

Interfaz	Para la interfaz usamos JavaFX en IntelliJ para enseñar el programa de una forma más visual	
IDE	IDE que se usó para realizar la lógica en Java	

7 Patrones de código utilizados

En esta práctica se utilizaron los dos siguientes patrones de código que facilitaron la organización y el flujo de información.

7.1 Patrón Productor - Consumidor

Es un patrón de diseño concurrente que sirve para distribuir trabajo en diferentes hilos donde los datos son pasados de una clase Productor a un (Buffer) que va a almacenar esos datos. Y pasa a otro hilo que es el Consumidor, que recolecta esa información cuando le interese y cuando pueda.

7.2 Patrón Vista - Controlador

Es un patrón de diseño que separa una aplicación en partes: una parte se encarga de conectar los datos y la lógica con la parte de la interfaz. Donde pasamos la información a través de un controlador.

8 Clases e Interfaces

Explicación breve de cada clase y interfaz y las herramientas necesarias para llevar a cabo la aplicación. La "base de datos" no existe, los datos están añadidos manualmente, por un lado los camareros y los clientes.

8.1 HelloController

- Declaramos variables:
 1. TextArea -> para el contenido principal de todo el flujo del ejercicio.
 2. BotonEmpezar -> al darle inicia la simulación y llama a la función "iniciar()".
 3. CamarerosArea -> espacio que va a guardar la información de los camareros, enseñando una lista.
 4. ClientesArea -> espacio que va a guardar la información de los clientes, enseñando una lista de clientes contentos y cuales no.
 5. BaristaArea y BufferArea -> Nos va mostrar la información de como está produciendo los cafés el barista y como está la cola de los cafés.
 6. Lista Camareros y Lista Clientes -> son los que se van a enseñar en las pantallas de la derecha de la interfaz, datos de los camareros y los clientes.
- Función al ejecutar el ejercicio, nos aparece un mensaje de darle a un botón para inicializar y no nos deja editar los elementos.
- Función "iniciar()" que funciona al pulsar en el botón "BotonEmpezar" y dentro de la función tenemos:
 1. La lista de datos de los camareros y la de los clientes.

2. llamamos a las funciones de las listas de los clientes y camareros para enseñarlas en las pantallas de la derecha.
 3. Iniciamos los camareros, barista, cola y con un new Thread, y con un botón manual para añadir los clientes entrando por tiempos a la cafetería y esperamos a que acaben. Eso todo lo ejecutamos ".start()".
- Funciones para actualizar el estado de los clientes, los camareros, barista y el buffer obteniendo los datos de las clases.
 - Funciones para listar los clientes y camareros, que nos los lista en las ventanas de la derecha de la interfaz.
 - Función para ir actualizando el estado del buffer a través de los cafés esperando a ser servidos y los cafés ya servidos.

8.2 Clases en Java

Al ejercicio anterior le tenemos que añadir 2 clases más y modificar la clase Camarero. Vamos a crear la clase Barista que va a equivaler al "Productor" que va a producir los cafés en este caso, una clase "Cola" que es en la que vamos a añadir los cafés que se están haciendo y quitando los mismos cuando el camarero los necesite. En la clase Camarero haremos que en vez de "servilo" el camarero, lo recoga de la cola y lo sirva.

8.2.1 Clase Barista

Lo que creamos es lo siguiente:

```
1 public class Barista extends Thread{
2     private Cola cola;
3     private HelloController controller;
4
5     public Barista(Cola cola, HelloController controller) {
6         this.col = cola;
7         this.controller = controller;
8     }
9
10    public void run(){
11        controller.agregarMensaje("Barista inicio su turno");
12        try{
13            //Crea 40 cafes como maximo por jornada
14            for(int i=1; i<40; i++){
15                //Meter los cafes hechos a la cola
16                cola.put();
17                Thread.sleep(3000);
18            }
19            controller.agregarMensaje("Barista ha terminado su turno");
20        }
21        catch (InterruptedException e){
22            e.printStackTrace();
23        }
24    }
25 }
26 }
```

- El barista es un hilo y tiene como atributos la cola y el controller para pasarle información a la interfaz.
- Constructor de la clase con los atributos.
- Función "run()" que se encarga de arrancar la jornada del barista, puede crear 40 cafés en total y los va añadiendo cada poco tiempo a la cola.
- A la interfaz la información que le pasa es que el barista inicia su jornada y cuando la acaba.

8.2.2 Clase Cola

Lo que creamos es:

```

1 public class Cola {
2     private int cafes;
3     private int capacidadMaxima = 40;
4     private HelloController controller;
5     private int preparadosTotal = 0;
6     private int servidosTotal = 0;
7
8     public Cola(int cafes, HelloController controller) {
9         this.cafes = cafes;
10        this.controller = controller;
11    }
12
13    public synchronized void put(){
14        try {
15            //Si la cantidad de cafes es igual o menos que la cantidad maxima
16            //pues el barista tiene que esperar que el camarero recoga cafes
17            while (cafes >= capacidadMaxima) {
18                agregarMensaje("El barista espera - Buffer lleno: " + cafes + "/" +
19                               capacidadMaxima );
20                wait();
21            }
22            //Si puede meter mas cafes los hace y nos suma cuantos tiene en total y
23            //notificamos a todos
24            Thread.sleep(500);
25            cafes++;
26            preparadosTotal++;
27            agregarMensaje("Barista prepara cafe, " + "lleva: " + cafes);
28            agregarMensaje("Total preparados: " + preparadosTotal);
29            controller.actualizarBuffer();
30            notifyAll();
31        }
32        catch (InterruptedException e) {
33            e.printStackTrace();
34        }
35    }
36
37    public synchronized void get(String camareroNombre){
38        try {
39            //Si la cantidad de cafes es 0, el camarero tiene que esperar al que el

```

```

39         //barista meta cafe
40         while (cafes == 0) {
41             agregarMensaje(camareroNombre + "espera - Buffer vacio");
42             wait();
43         }
44         //Si puede retirar lo retira y nos lo meta a cafes servidos
45         Thread.sleep(300);
46         cafes--;
47         servidosTotal++;
48         agregarMensajeCamarero("Cafe esta siendo recogido por el camarero");
49         agregarMensaje(camareroNombre + " recogio un cafe");
50         controller.actualizarBuffer();
51         notifyAll();
52     }
53     catch (InterruptedException e) {
54         e.printStackTrace();
55     }
56 }
57
58 //Para pasar los mensajes a un textArea despues
59 public void agregarMensaje(String mensaje){
60     Platform.runLater(() -> {
61         controller.agregarMensaje(mensaje);
62     });
63 }
64 //Para pasar los mensajes a otro textArea
65 public void agregarMensajeCamarero(String mensaje){
66     Platform.runLater(() -> {
67         controller.agregarLog(mensaje);
68     });
69 }
70
71 public synchronized int totalPreparados(){
72     return preparadosTotal;
73 }
74
75 public synchronized int totalServidos(){
76     return servidosTotal;
77 }
78
79 public synchronized int cafes(){
80     return cafes;
81 }
82 }

```

- La Cola va a tener como atributos: el número de cafés, capacidadMaxima, el controlador para pasar la información, cafés preparados y cafés servidos.
- Constructor de la clase con los atributos.
- Función "put()" que mientras los cafés sean mayor o igual que la capacidad máxima del buffer el barista espere, sino añadimos un café y mandamos la información que el Barista ha hecho el café y está listo para recogerlo y lo notificamos a todos.
- Función "get()" que mientras en la cola haya 0 cafés el camarero tenga que esperar y sino pues el camarero retira un café para entregárselo al consumidor, lo notificamos y mandamos la información que nos interese enseñar an la interfaz.

- Funciones de "agregarMensaje()" y "agregarMensajeCamarero()" que sirven para mandar la información a través del controlador.
- Tenemos otras 2 funciones que las usamos para devolver el total del cafés preparados y el total servidos para ir actualizando la información en la interfaz.

8.2.3 Clase Camarero

Lo único que vamos a implementar en la clase camarero es un atributo Cola que va a permitir recoger los cafés de la cola para servirlos.

- Modelo de datos con propiedades: nombre, estado, controlador para pasar la información y cola.
- Atributos:
 1. nombre -> nombre del camarero.
 2. ocupado -> buleano que se pone en "false" al entregar el café y empieza en "null".
 3. controller -> para pasar la información que queremos mostrar a la interfaz.
 4. cola -> nos permite acceder a la clase cola para recoger los cafés que va produciendo el barista.
- Constructor de la clase con los atributos.
- Función "prepararCafe()" que pasándole los clientes por parámetro y con un try catch gestiona:
 1. Los camareros a que cliente está preparando el café
 2. Aquí cogemos el café de la lista llamando a la función y pasándole el nombre del camarero con el que estamos trabajando.
 3. Esto lo pasa a la clase Cliente y marca el buleano "CafeEntregado" como "true".
- Función run que se ejecuta en el main con .start() y nos dice que los camareros están listo para atender.
- Getters y setters de todos los atributos.

8.2.4 Clase Cliente

De la clase Cliente no tenemos que realizar ninguna modificación porque no es ni productor ni consumidor en este caso.

- Atributos:
 1. nombre -> nombre del cliente.
 2. tiempoEspera -> cantidad de tiempo que está dispuesto a esperar el cliente a ser atendido.
 3. Lista Camareros -> lista de los camareros para ver cual está ocupado o no.

4. `cafeEntregado` -> booleano que se pone en true al entregar el café
- Constructor de la clase con los atributos.
 - Función `run` que se ejecuta en el main con `.start()`, que contiene un try catch en el encontramos:
 1. Los clientes van llegan a la cafetería
 2. Creamos una variable "ocupado" que inicia siendo "null" porque ninguno de los camareros está ocupado al principio.
 3. Recorriendo la lista de camareros buscamos un camarero libre y lo marcamos como ocupado. Y si está libre, llamamos la función "prepararCafé()" que se encuentra en la Clase camarero.
 4. Luego para ver si sigue estando en el tiempo de espera del cliente, busqué como calcular el tiempo desde que se inicia con "currentTimeMillis()" y se lo restamos al tiempo de inicio si es mayor que el tiempo de espera, el cliente se va, sino el cliente ha recibido el café y se ha ido.
 - Getters y setters de todos los atributos.
 - La clase Cliente tienen los mismos atributos y la misma estructura que la realizada en el ejercicio de Java anterior, lo nuevo es un atributo `HelloController controller` que va a pasar los datos que necesitamos de los clientes al `HelloController`.
 - Y en vez que imprimirlo con un "sout" lo pasamos por un (`controller."funcionParaPasarDatos"`), esos datos se pasan y los enseña en los `textArea`.
 - Los getters, setters y la función "run()" se mantienen igual.

8.2.5 Hello View (xml)

Todos los elementos visuales con los ids y las acciones vinculadas para que se ejecute todo correctamente en la interfaz.

- Tenemos un `AnchorPane` que envuelve a todos los elementos y es el que se conecta al controller `HelloController`.
- Hay tres fondos que se paran las pantallas, por un lado la ejecución, por otro los datos de Cliente y Camarero y por último tenemos al barista y a la cola de los cafés para servir y los servidos.
- Un `TextArea` con el `fx:id="texto"`.
- El botón `empezar` que inicializar el simulador.
- Dos `TextArea` que nos enseñan los datos de los clientes y los camareros.
- Tenemos una leyenda que nos indica que representa cada icono.
- Hay un `TextArea` que nos enseña como el Barista trabaja y va preparando cafés constantemente.

- Y otros dos textAreas que nos enseñan por un lado los cafés para servir y por otro los servidos.
- A parte hay un archivo "style.css" que tiene el estilo del botón, los fondos, etc.

Aquí vemos un ejemplo del funcionamiento del programa a través de la interfaz.

